00 — Start here: how to read every number on the screen

The other files in this folder answer *"is 200.60 correct?"*. This one answers *"why is it 200.60?"* — and tells you which file to open when you see a number you do not recognise.

Read this once. After that the per-scenario files make sense on their own.

Part 1 — The words, for someone who has never worked with ships

kW (kilowatt) — how hard something is working *right now*. Like horsepower. An engine rated 24 000 kW can produce at most that much at any instant.

kWh (kilowatt-hour) — how much work was done *in total*. kW × hours. A 1 000 kW battery running for 2 hours delivers 2 000 kWh. One is the size of the crane; the other is how much it lifted today.

> This distinction matters immediately: **a battery's `powerKw` is the only figure that enters the fuel calculation.** `capacityKwh` is used solely for a plausibility warning (can it sustain that power for 30 minutes?). Doubling the capacity changes no number on the results panel.

Propulsion — the power that moves the ship forward. **Hotel / mission** — everything else: lights, galley, air conditioning, pumps, cranes. The ship's "hotel".

ME — Main Engine. The big one, turns the propeller. **AE — Auxiliary Engine.** A separate diesel that makes electricity only. **SG — Shaft Generator.** A generator bolted to the main engine's shaft. Makes electricity while the ME is turning anyway — so it is much cheaper than starting a separate diesel. **This is why the calculation always fills the SG before starting an AE.**

SFOC — Specific Fuel Oil Consumption, in g/kWh: grams of fuel burned per kilowatt-hour of work. Lower is better. **It depends on how hard the engine is working** — a large 2-stroke burns about 167 g/kWh at 63 % load and over 230 g/kWh at 5 % load. Every engine model has its own curve, stored in the database.

DP — Dynamic Positioning. The ship holds one spot — over a wellhead, beside a platform — using thrusters that constantly push back against wind and current. It does not travel; it fights to stay still. That is why a DP mode has "required DP power" (thrust) instead of "propulsion".

Spinning reserve — engines kept running below their efficient load *just in case* the demand jumps. They are not doing useful work; they are insurance.

Peak shaving — flattening the top of a fluctuating load so the plant does not have to be sized for the peak.

Part 2 — The one idea that makes everything else obvious

A ship's engines are sized for the peak, not the average.

Demand is never steady. A wave hits, the crane starts, the wind gusts. If demand can jump by 573 kW at any moment, you must already have 573 kW of *running* engine able to absorb it. A diesel takes minutes to start; a wave takes seconds.

So ships run extra engines at low load, permanently, as insurance. And here is the expensive part:

> **A diesel at 15 % load burns far more fuel per unit of work than the same diesel at 60 %.**

A battery responds in milliseconds. Whatever the battery can absorb, you no longer need a running engine standing behind it. **That is the entire product.**

Everything in these walkthroughs is an elaboration of that one sentence.

> **The same machine is drawn in `00a-PIPELINE-DIAGRAM`, with scenario 01's numbers flowing through it.** Steps 1–2 are figure 1, steps 3–4 are figure 2, step 7 is figure 3. If a step below does not land, look at its picture.

Part 3 — The seven steps, in plain language

Every scenario follows the same pipeline. The formulas are in `README.md`; this is what each step *means*.

Step 1 — The battery cascade: who gets the battery's power

The battery has a power budget (its `powerKw`). It is handed out to the loads **in strict priority order**, and the order is data, not code — it lives in `appsettings.json`:

```
DpReserve  →  DpDemand  →  Mission  →  Propulsion  →  Hotel
```

Each row is described by four numbers:

	Meaning
H	how much this row *wants* covered
I	how much it actually *takes* from the remaining budget — `min(budget, H)`
J	how much that take *counts as covered* — `I × CoverageFactor`
L	what is left for the diesels to carry — `H − J`

H is computed three different ways, depending on what kind of load it is:

Row type	H =	Why
Peak shaving (Propulsion, Hotel)	average × VariationFactor	only the fluctuating band needs shaving — 5 % of propulsion, 2 % of hotel
Mission (a crane)	the machine's full rating	it can start at *any* moment, so its whole draw is a potential peak
Reserve (DP redundancy)	the full requirement	a class rule demands that much *readiness*; there is no "35 % of a rule"

CoverageFactor is the number people get stuck on. It is a modelling assumption: *what fraction of this load's swing can a battery realistically catch?*

Row	CoverageFactor	Reading
DpReserve	**1.00**	a 400 kW battery satisfies a 400 kW readiness rule, kW for kW
DpDemand, Mission	0.50	half of the swing is catchable
Propulsion	0.35	only about a third — the rest is too fast, too large or too sustained
Hotel	0.05	almost none

So when a card says "*573.15 × 0.35 = 200.60*", the 0.35 is not physics. It is a business assumption that lives in a config file, and changing it changes every savings figure in the app.

The invariant to hold on to: `J + L = H`. **The sea decides the total swing; the battery only decides who carries it.** That is why Peak Shaving + Spinning Reserve is always the same number for a given ship, no matter how big the battery.

Step 2 — Whatever the battery did NOT cover becomes real demand

`propulsion' = propulsion + L(propulsion-side rows)` `hotel' = hotel + L(hotel-side rows)`

The uncovered swing has to be carried by running engines, so it is added to the demand.

Covered power (J) is never subtracted from demand. A covered reserve is *readiness*, not load — nothing is consumed. That is why in scenario 04 the 400 kW of DP redundancy appears nowhere in Power Demands once the battery covers it.

Step 3 — Spreading the demand over the machinery

For each candidate combination of (number of MEs running, SG on/off, number of AEs running):

```
SG power = min(hotel', SG capacity of the RUNNING MEs)    the SG is filled first
AE power = hotel' − SG power                              the AEs take the remainder
ME power = propulsion' + SG power                         the SG is a load ON the main engine
load %   = power / capacity of the running units
```

Two consequences worth remembering:

- **SG capacity scales with running MEs.** Two MEs running means 2 × 3 250 = 6 500 kW of shaft generator — often enough to carry the whole hotel and leave the AEs switched off.
- **The ME carries the SG's output on top of propulsion.** That is why the ME figure is always larger than the propulsion figure.

Step 4 — From load to fuel

```
FOC [t/h] = ME power × SFOC_ME(ME load %) / 1e6
          + AE power × SFOC_AE(AE load %) / 1e6
```

SFOC is read off the engine's curve by linear interpolation. **This is the step where "running an extra engine at low load" becomes expensive** — the curve, not the arithmetic, does the work.

Step 5 — Ranking, and the two baselines

All valid combinations are sorted by fuel consumption.

- **Optimal** = the cheapest. This *is* Integration Level 1. It is not user-selectable.
- **Baseline** = what the ship is assumed to do *today*, without a smart system. Savings are measured against it.

The baseline rule differs, and this surprises people:

```
no battery   →  the WORST combination        (sorted.Count − 1)
battery on   →  the third from worst          Math.Max(0, sorted.Count − 3)
```

`Math.Max(0, …)` matters: when only two combinations survive, `2 − 3` clamps to **0**, and the baseline becomes the optimum itself — **so that mode reports zero Level 1 savings.** Scenario 04's DP mode does exactly this.

The radio buttons in "Assumed Configuration" override the baseline. Doing so re-runs the calculation silently and shows a "*Custom baseline*" hint.

> **Known defect:** the client's check for "is this the default baseline?" assumes the no-battery rule (`count − 1`). With a battery active the backend picks `count − 3`, so **every battery scenario shows the orange "Custom baseline" hint even when nothing was chosen.** Compare scenario 01 (battery, hint shown wrongly) with scenario 03 (no battery, no hint).

Step 6 — Annual figures

```
FOC t/yr = FOC t/h × mode hours,  summed over all active modes
CO2      = FOC × the fuel's own factor    (MGO/MDO 3.93267 · HFO 3.114 · LNG 2.753)
cost     = FOC × fuel price
savings  = baseline − optimal
```

ME and AE each use **their own fuel's** CO2 factor — a vessel burning LNG in the main engine and MGO in the auxiliaries produces two different splits that must sum to the panel total.

When several modes are active, the Power Demands header shows an **hours-weighted average**, not a sum: total energy ÷ total hours.

Step 7 — The Battery Benefit (the green badge)

This is a *different question* from Level 1 savings, and it is computed by running the whole pipeline twice:

	Demand	Meaning
World A	average + L (uncovered only)	the ship as configured, with its battery
World B	average + H (the full swing)	the same ship, if the diesels carried everything

```
Benefit = (FOC_B optimal − FOC_A optimal) × hours,   summed per mode
```

Both figures are optima. The counterfactual deliberately ignores any pinned baseline, or the comparison would stop being optimal-vs-optimal.

A useful approximation: `Benefit ≈ Peak Shaving × SFOC × hours`. In scenario 01 that gives `204.4 kW × 167 g/kWh × 5 000 h / 1e6 = 170.7 t/yr` against the true 173.7. The benefit is very nearly **linear in Peak Shaving** — which is why the CoverageFactor is the most consequential setting in the whole file.

Part 3b — When the ship does more than one thing

Steps 1–5 above describe **one mode**. A vessel that sails for 5 000 hours and then holds position for 2 000 is doing two different jobs, and the pipeline is run **once for each of them**.

This is the single most common source of confusion, because the screen does not make it obvious.

What happens per mode, and what does not

Runs separately for **every** active mode	Runs for **Transit only**	**Summed** across modes
the battery cascade (steps 1–2)	Level 2	FOC in t/yr
the demand split over SG / AE / ME (step 3)	Level 3	CO2
the list of valid combinations	the pinned-baseline radio	cost
its own baseline and its own optimum (step 5)	the combination table you see on screen	
its own FOC in **t/h** (step 4)		

The Transit-only column is a deliberate decision (D4/Q5): *Level 2 has no counterpart in the reference workbook outside Transit*, so it is not computed there. In code the other modes receive an empty `Level2Result` and `Level3Result`, and a `null` baseline index.

The arithmetic this implies

```
per mode:   own demand → own combinations → own baseline, own optimum → own t/h
at the end: Σ (mode's t/h × mode's hours)
```

There is no single "tonnes per hour" for the vessel. Scenario 04's Transit burns 2.578548 t/h; its DP burns 0.703744 t/h, because DP's main engine carries 4 028 kW instead of 14 713. Multiplying Transit's rate by DP's hours over-states the year by about 3 750 tonnes.

Why savings can look like they come from one mode

Savings are a **difference**, and each mode appears on both sides of it:

```
                baseline      optimal      difference
Transit         13 127.3     12 892.7        234.6
DP               1 407.5      1 407.5          0
                ────────     ────────       ──────
                14 534.8     14 300.2        234.6
```

DP is not excluded — it is present in both totals with the same figure, so it cancels. It contributes nothing because **its baseline equals its optimum**, which happens whenever the clamp in step 5 bites.

Scenario 04 and scenario 11 are mirror images of each other, and reading them as a pair is the fastest way to understand this:

	Battery applies to	Which mode clamps	Level 1 savings come from
04	DP	DP	**Transit only**
11	Transit	Transit	**the other four modes only**

Same rule, opposite outcome. There is nothing privileged about Transit in Level 1 — only in Levels 2 and 3.

> A consequence worth holding on to: **a battery in a mode can suppress that mode's Level 1 savings.** The battery switches the baseline rule from "worst" to "third from worst"; on a short list that clamps to the optimum. The battery's value has not vanished — it has moved to the green Battery Benefit badge instead.

Three different "load %" on one screen

They come from three places in the code and answer three different questions:

Where	Whose load	Source
Baseline panel, "Average Load"	**Transit's baseline** combination	`Level1DetailsBuilder`
Assumed Configuration table, per row	**Transit**, one row per combination	`Level1DetailsBuilder`
Integration level cards, "Average Load"	**Transit's optimal** combination	`TierResultBuilder`

Scenario 01 shows 62.9 % in the Baseline panel and scenario 03 shows 63.7 % in the level card — different because one is what the ship does today and the other is what it could do.

A fourth figure, weighted by hours across **all** modes, is computed in `PowerDemandsBuilder` (`CalculateWeightedLoadPercent`) and drives the Power Demands header — that one is not a Transit figure.

Part 4 — The three worlds (and the trap)

The pipeline can be run against three different demands, and only two of them are used by the Benefit. Confusing them costs an afternoon.

World	Who carries the swing	Scenario 01 figures
A — battery on	battery 204.4, diesels 444.7	propulsion 11 835.5 · hotel 3 872.2 · **ME 15 086**
B — the Benefit's reference	diesels carry **all** 649.15	propulsion 12 036.15 · hotel 3 876 · **ME 15 286**
C — battery unticked in the UI	**nobody**	propulsion 11 463 · hotel 3 800 · **ME 14 713**

Scenario 03 *is* world B written down as its own scenario. Its inputs are scenario 01's raw loads plus the full swing. That is why the hand-check works:

```
03 Integration level 1:  13 459.9 t/yr
01 Integration level 1:  13 286.2 t/yr
                         ─────────
                            173.7 t/yr   =  01's Battery Benefit
```

The trap: unticking "Enable Battery" in the UI produces world **C**, not world B. No battery means no cascade at all, so the swing is not carried by anyone — a plant that burns *less* than the battery case. Subtracting C from A gives a negative number and makes the battery look harmful.

The code names this explicitly:

> *"Note this is NOT the same as passing no battery adjustment at all. That would model a plant which does not carry the variation on either side — a fiction that burns LESS fuel than the battery case and would make the benefit come out negative."*

To check a Benefit by hand, load scenario 03 — do not untick the battery.

Part 5 — Which numbers are physics and which are decisions

The most useful lens when a figure looks wrong: *can this be changed without touching code?*

Number	Where it lives	Changeable
Variation factors (propulsion 5 %, hotel 2 %)	`appsettings.json`	**configuration**
Coverage factors (1.00 / 0.50 / 0.35 / 0.05)	`appsettings.json`	**configuration**
The cascade's priority order	`appsettings.json`	**configuration**
Fuel prices, CO2 factors, vessel DRC variations	`appsettings.json`	**configuration**
Tier investments ($110k / $140.5k / $210k)	`appsettings.json`	**configuration**
SFOC curves per engine	database	manufacturer data
"Third from worst" baseline rule	`Level1OptimizationService`	**code only**
Which cascade rows exist per mode	`BatteryAllocationService`	**code only**

The last two are the inconsistency worth knowing about: the cascade itself is fully configurable, but two rules that shape its result are not.

Part 6 — Reverse index: "I see this number, where does it come from?"

On screen	Produced by	Formula	Cleanest example
Spinning Reserve	Step 1	Σ L over all rows	01
Peak Shaving	Step 1	Σ J over peak-shaving rows **only**	04 (shows why the reserve is excluded)
A row's **Variation (kW)**	Step 1	H — three formulas, see Step 1	05 (mission), 04 (reserve)
A row's **Battery Used**	Step 1	I = min(remaining budget, H)	02 (budget runs out mid-cascade)
A row's **Covered ±**	Step 1	J = I × CoverageFactor	01
Battery Benefit	Step 7	(FOC_B − FOC_A) × hours	01, verified against 03
Propulsion Power row	Step 2	propulsion + uncovered propulsion-side L	01
Hotel/Mission row	Step 2	hotel + uncovered hotel-side L	04
Shaft Generators column	Step 3	min(hotel', SG capacity × running MEs)	03 (2-ME row: SG doubles)
Auxiliary Generators column	Step 3	hotel' − SG power	01
Main Engine header	Step 3 + 6	propulsion' + SG power, hours-weighted over modes	04 (two modes)
Engine Energy Demands	Step 6	power × mode hours	04
Average Load %	Step 3	power ÷ capacity of *running* units	03
FOC (ton/hr) in the combination table	Step 4	Σ power × SFOC(load) / 1e6	01
Which row is **selected** in Assumed Configuration	Step 5	`count − 1`, or `max(0, count − 3)` with battery	01 vs 03
The rows in **Assumed Configuration** at all	Step 5	Transit's combinations — never any other mode's	04 (two modes, one table)
Why a mode contributes **no** savings	Step 5 + Part 3b	its baseline equals its optimum, so it cancels in the difference	04 (DP), 11 (Transit)
A **second row** in a Power Demands table	Part 3b	one row per active mode; the header is hours-weighted	04, 11
Baseline FOC ton/yr	Step 6	baseline t/h × hours	01
Integration level 1 savings	Step 5 + 6	baseline − optimal	03
Level 2 contribution	Level 2	redistributes hotel between SG and AEs	**19** (the only place it is non-zero)
Level 3 contribution	Level 3	DRC damping of the hotel swing	**11** (32.2 t) · 14 (vessel-type lookup)

On screen	Produced by	Formula	Cleanest example
CO2 ton/yr per engine	Step 6	FOC × that engine's fuel factor	13 (two different fuels)
Fuel Cost / Cost Savings	Step 6	FOC × fuel price	01
Payback / ROI	Step 6	fixed tier investment ÷ annual saving	01
A red **400 error**	validation, before Step 1	plant cannot carry the load	17
A **PTI gate** refusal	Step 3 gate	battery needs more shaft-motor capacity than exists	09

Part 7 — A reading order that covers the whole machine

You do not need all 35. Six scenarios exercise every mechanism; the rest are the same mechanisms with different inputs.

Order	Scenario	The one thing it teaches
1	**01**	the full cascade, both tiles, the Benefit
2	**02**	what happens when the budget runs out mid-cascade
3	**03**	no battery — and it *is* world B of 01's Benefit
4	**04**	two modes at once · Reserve ≠ Peak Shaving · the baseline clamp
5	**19**	**Level 2 and Level 3 actually doing something**
6	**11**	all five modes · Level 3 from the vessel-type lookup
7	**05 + 06**	the Mission row, then the same crane devouring the whole budget
8	**09**	how the app refuses to calculate, and why
9	**12**	sail — an intervention *before* the cascade

Optional, two minutes: **17**, a plant that cannot carry its load. It fails in validation, before Level 1 ever runs — a different failure path from 09.

Scenario 04 is the hardest of the first four because it introduces three new things at once. If it does not click, take it apart in the app instead of re-reading:

1. Set DP hours to 0 → only Transit remains, the battery panel disappears entirely
2. Set Transit hours to 0 → only DP remains; **the Battery Benefit stays 139.9**, proving the modes are independent sums
3. Put both back → the totals are the sum of the two
4. Set DP redundancy from 400 to 0 → **the tiles do not move at all, but the Benefit collapses** — which is exactly what the Reserve row does

---

Part 8 — How much to trust each number

Scenarios	Status
01–18	verified against the reference workbook `PowerPlantSetupAdvisesIncludingPTIOAndbatteries_test.xlsx`. These are proof.
19–35	**characterisation snapshots generated from the code.** They detect change; they do **not** prove correctness. A figure marked "pending reference verification" has never been checked against anything outside the application.

The distinction is not pedantry. Finding 5 (`baselineIndex` lost on restore) was recorded as **FIXED** on 2026-08-03 and was not actually fixed until 2026-08-05 — it was believed rather than tested.

Where the numbers disagree, the Excel workbook wins, not the code.

Part 9 — Open findings that affect numbers

Do not treat these as bugs to fix while reading; they are known, logged, and awaiting a product decision.

1. **SG-forced rule runs a main engine in Port/Anchor.** Changes results.
2. **The DP weather factor is never applied.** A missing feature, not a rounding issue.
3. `SfocService` **catches every exception and returns 220 g/kWh.** A database problem therefore produces a plausible but wrong fuel figure instead of an error. The most serious of the four for anyone quoting a savings number to a customer.
4. `Level3Result.VariationPerGeneratorKw` **is misnamed** — the value is the bus-wide swing, not a per-generator figure.

Plus the client-side "Custom baseline" defect described in Step 5.

Numbers in this folder were verified against a live API built from commit `559aef2`. The backend has since been refactored, but every change was proven behaviour-preserving by 18 frozen golden snapshots compared byte-for-byte — so the arithmetic below is unchanged. See `docs/refactoring/backend-refactor-design.md`.
